Simple Dictionary — Requirements Document

DRAFT — complete the blank fields and obtain team approval before submission.

Item	Details
Course / year	BSIT-SD, 2nd year
Section	IDB1-SD
Subject / coverage	MOBAPP1, Modules 6–9
App	Simple Dictionary
Repository	https://github.com/paulAngelo-ws/dictionary-2.0.git
Group / instructor	_____ / _____
Submission / frozen commit	_____ / _____
Prepared / reviewed by	_____ / _____

1. Team and responsibilities

The team reports one Developer, one BA, and two QA members. The exam specifies three members: BA/Developer, Dedicated Developer, and QA. Instructor confirmation of the four-member arrangement is pending. The following are proposed responsibilities, not claims about completed contributions. All members should write or review code and understand the complete application.

Member name	Role	Proposed responsibility	Actual contribution / commit evidence
_____	Developer	Shared architecture, networking, storage, integration and setup instructions	To be confirmed
_____	BA	Requirements, user stories, screen flow, acceptance criteria and UI implementation/review	To be confirmed
_____	QA 1	Desktop functional tests, defect	To be confirmed

Member name	Role	Proposed responsibility	Actual contribution / commit evidence
_____		reporting and code/test review	
_____ _____	QA 2	Android tests, persistence/offline tests and automated-test review	To be confirmed

2. Purpose and intended users

Simple Dictionary helps students and other English-language users look up formal definitions and internet slang in one application. Users can revisit saved searches, mark favorites and choose whether slang appears expanded by default. Slang can contain explicit language; it is collapsed by default unless the user changes the setting.

The scope is a small Kotlin Multiplatform application with three main screens: Input, Result and History. Android is the required examination platform. The project retains an iOS source set and also includes desktop and browser targets. Only desktop manual testing has been reported so far; this does not establish that each feature passed.

3. Features and scope

ID	Requirement
FR-01	Search for a nonblank word or phrase and display available definitions.
FR-02	Show formal definitions and slang with their source, and a part of speech or example when supplied.
FR-03	Show a spelling suggestion when Merriam-Webster returns one; allow searching it.
FR-04	Provide Input, Result and History navigation and pass the selected word to Result.
FR-05	Represent initial, loading, success and error conditions in the UI.
FR-06	Save successful searches and definitions locally and restore them after restarting.
FR-07	Allow saved words to be opened offline; attempt a background refresh when opened.
FR-08	Persist favorites, show them first in history, and preserve their flag during refreshes.

ID	Requirement
FR-09	Save the Show slang by default setting separately from dictionary data.
FR-10	Provide a manual refresh action and retain useful saved data when the refresh fails.

The app does not include user accounts, cloud synchronization, pronunciation playback, a full offline dictionary, history deletion or custom definition editing. Offline lookup is limited to previously saved words. API availability, API-key validity and browser CORS policies are external dependencies.

4. User stories and acceptance criteria

Each acceptance criterion has a matching manual test in the QA document.

Story / criterion	User story	Acceptance criterion
US-01 / AC-01	As a user, I want to look up an English word.	Given connectivity and a responding dictionary, searching hello opens Result with the same word and at least one definition labelled with its source.
US-02 / AC-02	As a user, I want accidental blank searches ignored.	Searching an empty or whitespace-only input sends no lookup and creates no history record.
US-03 / AC-03	As a user, I want to know the search status.	A request in progress shows Loading. A failed uncached lookup shows a readable error without crashing.
US-04 / AC-04	As a user, I want to correct a misspelling.	When the API supplies a spelling suggestion, tapping it opens Result for the suggested word. Suggestions alone are not saved as definitions.
US-05 / AC-05	As a user, I want to move between screens.	Input, Result and History are reachable. Selecting a history word opens the matching Result. Android Back returns through the navigation stack.
US-06 / AC-06	As a user, I want searches kept after closing.	After a successful lookup and a full app restart, History contains the word and its saved definitions can be reopened.
US-07 / AC-07	As a user, I want saved words without Internet.	After saving a word, disabling connectivity and reopening it still

Story / criterion	User story	Acceptance criterion
		displays its saved definitions; failed background refresh does not replace them with an error screen.
US-08 / AC-08	As a user, I want favorite words easy to find.	Starring a saved word moves it into the favorite group. Its flag survives a restart and a successful definition refresh. Unstarring updates history.
US-09 / AC-09	As a user, I want my slang-display choice remembered.	The setting defaults to off on fresh app data. Changing it, restarting and opening a slang result preserves the choice.
US-10 / AC-10	As a user, I want updated definitions when available.	Refresh requests online data. Successful results are saved; a failed refresh of a displayed successful result leaves that result usable.
US-11 / AC-11	As a user, I want newer searches to take priority.	When searches are started in quick succession, a previous request cannot replace the latest requested word's result.
US-12 / AC-12	As an Android user, I want a working installation.	The submitted project installs and launches on the recorded Android device/emulator and completes search, history, favorite and preference flows without a crash.

5. Screen flow and wireframes

These text wireframes describe the existing screen functions; they are not screenshots or proof of test execution.

Input

```
+-------------------------------------+
| Simple Dictionary                   |
| Look up definitions & slang          |
| [ Enter a word or phrase          ] |
| [ ] Show slang by default            |
| [ Search                          ] |
|                                     |
| [Input]       [Result]       [History] |
+-------------------------------------+
```

Result

```
+-------------------------------------+
| Selected word                       |
| [ Refresh definitions ]             |
| Loading / error / definitions        |
| Formal definition and source        |
| Optional spelling suggestion        |
| Slang notice [Show / Hide]           |
| Optional slang definition / example  |
| [Input]       [Result]       [History] |
+-------------------------------------+
```

History

```
+-------------------------------------+
| Search history                      |
| word + definition snippet        [*] |
| word + definition snippet        [ ] |
| Favorites appear first              |
| Tap a word to open its result        |
| [Input]       [Result]       [History] |
+-------------------------------------+
```

Main flow: Input → enter word → Search → Result → History → select saved word → Result. A spelling suggestion opens a new Result for the corrected word. The slang setting is changed on Input. Empty History and Result screens explain that no saved search/result is available.

Android deep linking is an additional implemented feature: dictionary://word/hello targets the word result. Native iOS/desktop URL registration is outside the current scope.

6. Architecture and module coverage

Module	Application
6 — Architecture	commonMain contains data, domain, presentation and UI code. Repository interfaces hide storage/network details. Use cases and constructor injection connect the layers. Platform source sets supply networking engines and storage locations.
7 — State	A shared ViewModel-like class exposes StateFlow for results, history and preferences. Sealed UiState models Idle, Loading, Success and Error. SharedFlow carries one-time messages; coroutines perform asynchronous work.
8 — UI/navigation	Shared Compose screens collect state. NavHost provides routes and a back stack; result/{word} carries the selected word.
9 — Networking/storage	Ktor and serializable models handle APIs, status checks and failures. Room persists definitions/history/favorites. DataStore persists the slang setting. Cached words display before background refresh.

Room is chosen for the structured saved-search data. SQLDelight is an alternative discussed in the module, not a second required database. Browser SQLite worker infrastructure is isolated in browserStorage; the retained third-party license and source attribution are documented in the README.

7. Approval and open items

Before submission, confirm the team-size arrangement, fill names and actual contributions, execute the manual QA cases, add Android device/emulator details and evidence, review the API access configuration, and identify the final frozen commit. Each team member should be able to explain the submitted code in their own words.

BA approval: _____ Date: _____ Developer review: _____ QA review: _____